

Séries Temporelles pour le Machine Learning

De la théorie aux modèles d'ensemble (Random Forest, XGBoost, LightGBM, CatBoost)

Support de cours

Étude de cas : anticipation de la demande transfusionnelle au Togo
(*projet LAFlè IA*)

Préparé pour	SAKPANI Paguiyedou
Formation	Licence Fondamentale IA & Big Data — ENP (ex-EPL)
Niveau requis	Régression linéaire/logistique, arbres de décision, KNN
Jeu de données	demande_sang_togo.csv (fourni avec ce cours)

Septembre 2026

Table des matières

1	Introduction : pourquoi les séries temporelles ?	2
2	Définitions fondamentales	2
3	Le jeu de données : demande transfusionnelle à Lomé	3
3.1	Chargement et premier coup d'œil	4
4	Décomposition d'une série temporelle	4
5	Stationnarité : le test ADF	5
6	Autocorrélation : ACF et PACF	5
7	Feature engineering temporelle	7
8	Validation croisée pour les séries temporelles	8
9	Passer aux modèles d'ensemble	8
10	Exercices de synthèse	10
11	Pour aller plus loin	10

1 Introduction : pourquoi les séries temporelles ?

Jusqu'ici, tu as appris des modèles qui traitent chaque observation comme **indépendante** des autres : dans le dataset Titanic, la survie d'un passager ne dépend pas de celle du passager précédent. Mais dans énormément de problèmes réels — et en particulier dans ton projet **LAFIè IA** — l'ordre du temps change tout : la demande de sang *aujourd'hui* dépend de ce qui s'est passé *hier*, *la semaine dernière*, et *à la même période l'année dernière*.

Pourquoi commencer par les séries temporelles avant Random Forest / XGBoost ?

Tu as raison de vouloir apprendre les séries temporelles en premier. En pratique, la façon la plus efficace de traiter un problème temporel avec des modèles comme Random Forest ou XGBoost n'est pas d'utiliser un algorithme « spécial », mais de **transformer la série en tableau de features** (valeurs passées, moyennes glissantes, informations calendaires), puis d'appliquer un modèle de ML classique dessus. Ce cours te donne exactement ces briques.

Ce cours utilise un jeu de données **synthétique mais réaliste**, directement inspiré de ton projet : l'indice quotidien de demande transfusionnelle à Lomé entre 2021 et 2025, influencé par la météo, le paludisme, les accidents de la route et les accouchements.

2 Définitions fondamentales

Définition — Série temporelle

Une série temporelle (*time series*) est une suite de valeurs numériques mesurées à des instants successifs, généralement à intervalles réguliers (chaque jour, chaque heure, chaque mois). On la note souvent $y_1, y_2, \dots, y_T$, où y_t est la valeur observée à l'instant t .

Définition — Fréquence

La fréquence est l'intervalle de temps entre deux observations successives (journalière, hebdomadaire, mensuelle...). Dans notre dataset, la fréquence est **journalière**.

Définition — Tendance (*trend*)

Évolution lente et globale de la série sur le long terme (croissance, décroissance, ou stabilité). Dans notre cas : la demande de sang augmente légèrement d'année en année (croissance démographique, usage hospitalier accru).

Définition — Saisonnalité (*seasonality*)

Motif qui se répète à intervalles réguliers et connus. On distingue plusieurs échelles :

- **Saisonnalité annuelle** : pic de paludisme pendant la saison des pluies, chaque année.
- **Saisonnalité hebdomadaire** : moins d'activité hospitalière programmée le week-end.

Définition — Résidu / bruit

Ce qui reste une fois qu'on a retiré la tendance et la saisonnalité : la part « imprévisible » de la série (variations aléatoires, événements ponctuels).

Définition — Stationnarité

Une série est dite **stationnaire** si ses propriétés statistiques (moyenne, variance) ne changent pas dans le temps. Beaucoup de modèles statistiques classiques (ARIMA) exigent la stationnarité. On la vérifie avec le **test de Dickey-Fuller augmenté (ADF)** :

- $p\text{-value} \leq 0.05 \Rightarrow$ on rejette l'hypothèse de non-stationnarité $\Rightarrow$ la série est **stationnaire**.
- $p\text{-value} > 0.05 \Rightarrow$ la série est **non stationnaire** (elle a probablement une tendance).

Définition — Autocorrélation

Corrélation d'une série avec elle-même, décalée de k pas de temps (appelés **lags**). Elle mesure à quel point le passé « explique » le présent.

- **ACF** (*AutoCorrelation Function*) : corrélation totale entre y_t et y_{t-k} , y compris les effets indirects via les lags intermédiaires.
- **PACF** (*Partial ACF*) : corrélation *directe* entre y_t et y_{t-k} , une fois retirée l'influence des lags intermédiaires. Très utile pour choisir quels lags garder comme features.

Définition — Lag (retard)

Un *lag* de k jours désigne la valeur de la série k jours avant l'instant courant : y_{t-k} . Une **lag feature** est une colonne du dataset contenant cette valeur passée, utilisée comme variable explicative.

Définition — Fenêtre glissante (*rolling window*)

Statistique calculée sur les w dernières observations avant l'instant courant (moyenne, écart-type, min, max). Exemple : la moyenne glissante sur 7 jours au temps t est la moyenne de $y_{t-7}, \dots, y_{t-1}$.

3 Le jeu de données : demande transfusionnelle à Lomé

Le fichier `demande_sang_togo.csv` contient **1826 lignes** (une par jour, du 1^{er} janvier 2021 au 31 décembre 2025) et les colonnes suivantes :

Colonne	Description
<code>date</code>	Date de l'observation (format AAAA-MM-JJ)
<code>temperature_c</code>	Température moyenne du jour (°C)
<code>precipitations_mm</code>	Précipitations journalières (mm)
<code>cas_paludisme</code>	Nombre de cas de paludisme déclarés
<code>nb_accidents</code>	Nombre d'accidents de la route
<code>nb_accouchements</code>	Nombre d'accouchements enregistrés
<code>demande_sang</code>	Cible : indice de demande transfusionnelle

Remarque

Ce jeu de données est **synthétique** (généré par simulation) mais construit pour reproduire des mécanismes réalistes : le paludisme suit les précipitations avec un décalage d'environ 3 semaines, les accidents sont plus fréquents le week-end, et il existe une tendance de fond plus quelques « chocs » simulant de petites épidémies. Il est fait pour t'entraîner avant de

travailler sur tes vraies données.

3.1 Chargement et premier coup d'œil

Listing 1: Chargement du dataset avec pandas

```
1 import pandas as pd
2
3 # parse_dates=["date"] : indique a pandas de convertir directement
4 # la colonne "date" en objet datetime (indispensable pour manipuler le
   temps)
5 df = pd.read_csv("demande_sang_togo.csv", parse_dates=["date"])
6
7 # Toujours trier par date : un CSV n'est pas toujours dans le bon ordre,
8 # et beaucoup d'operations temporelles (lag, rolling) supposent un ordre
   croissant
9 df = df.sort_values("date").reset_index(drop=True)
10
11 print(df.head())
12 print(df.info()) # types de colonnes, valeurs manquantes
13 print(df["demande_sang"].describe()) # statistiques descriptives de la
   cible
```

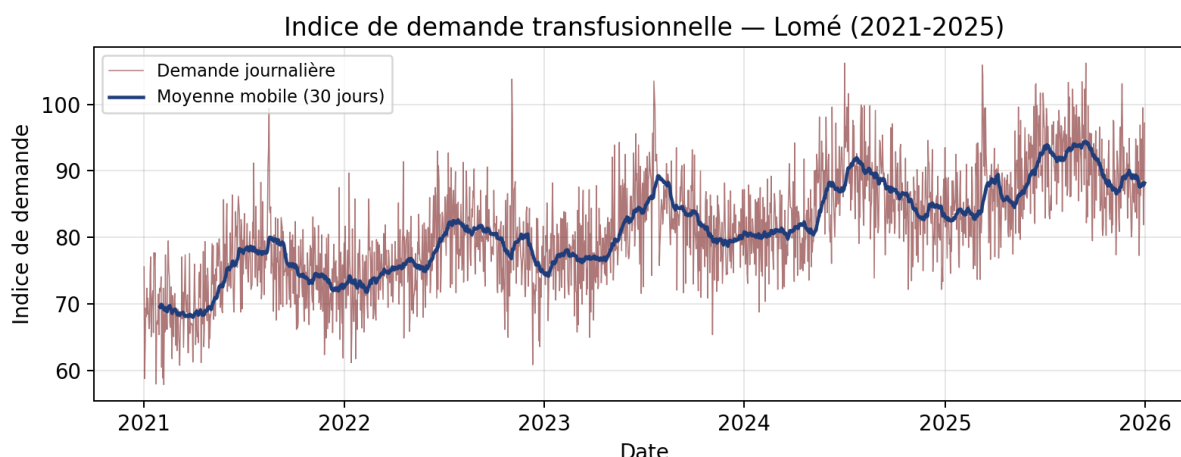


Figure 1: Série brute de la demande transfusionnelle (2021-2025) et sa moyenne mobile sur 30 jours. On distingue déjà une légère tendance croissante et des pics ponctuels.

4 Décomposition d'une série temporelle

Décomposer une série, c'est la séparer en trois composantes :

$$y_t = \underbrace{T_t}_{\text{tendance}} + \underbrace{S_t}_{\text{saisonnalité}} + \underbrace{R_t}_{\text{résidu}} \quad (\text{modèle additif})$$

On utilise un modèle **additif** quand l'amplitude de la saisonnalité reste stable dans le temps, et **multiplicatif** ($y_t = T_t \times S_t \times R_t$) quand elle grossit avec le niveau de la série (ex: ventes qui augmentent en volume ET en amplitude saisonnière).

Listing 2: Décomposition saisonnière avec statsmodels

```

1 from statsmodels.tsa.seasonal import seasonal_decompose
2
3 # On travaille sur un index temporel (obligatoire pour seasonal_decompose)
4 serie = df.set_index("date")["demande_sang"]
5
6 # period=365 : on indique la longueur du cycle saisonnier attendu
7 # (ici, un cycle annuel car les donnees sont journalieres)
8 # model="additive" : on suppose que la saisonnalite garde une amplitude
   stable
9 #
10 # ASTUCE : decompose sur la serie COMPLETE plutot que sur un nombre de
11 # cycles exactement entier (ex : exactement 2 x 365 jours). Avec trop peu
12 # de cycles complets, la fonction peut produire des residus faussement
13 # quasi nuls -- un piege frequent quand on debute.
14 decomposition = seasonal_decompose(serie, model="additive", period=365)
15
16 decomposition.plot() # trace automatiquement observee / tendance / saison
   / residu

```

5 Stationnarité : le test ADF

Listing 3: Test de Dickey-Fuller augmenté

```

1 from statsmodels.tsa.stattools import adfuller
2
3 resultat = adfuller(df["demande_sang"])
4
5 # resultat[0] : statistique du test (plus elle est negative, plus c'est
   stationnaire)
6 # resultat[1] : p-value -> c'est LA valeur a regarder
7 print("Statistique ADF :", round(resultat[0], 3))
8 print("p-value :", round(resultat[1], 4))
9
10 if resultat[1] > 0.05:
11     print("Serie NON stationnaire (presence d'une tendance)")
12 else:
13     print("Serie stationnaire")

```

Sur notre dataset, on obtient une **p-value** ≈ 0.24 , donc la série **n'est pas stationnaire** — ce qui est cohérent avec la tendance croissante qu'on a simulée.

Remarque

Pourquoi s'en soucier si on utilise Random Forest/XGBoost et pas ARIMA ? Parce que même avec des modèles d'ensemble, une tendance forte et non capturée par tes features peut biaiser tes prédictions (le modèle ne peut pas « extrapoler » une tendance qu'il n'a jamais vue). La solution : inclure des features qui capturent explicitement cette tendance (numéro du jour depuis le début, année, moyenne glissante longue).

6 Autocorrélation : ACF et PACF

Listing 4: Tracer l'ACF et la PACF

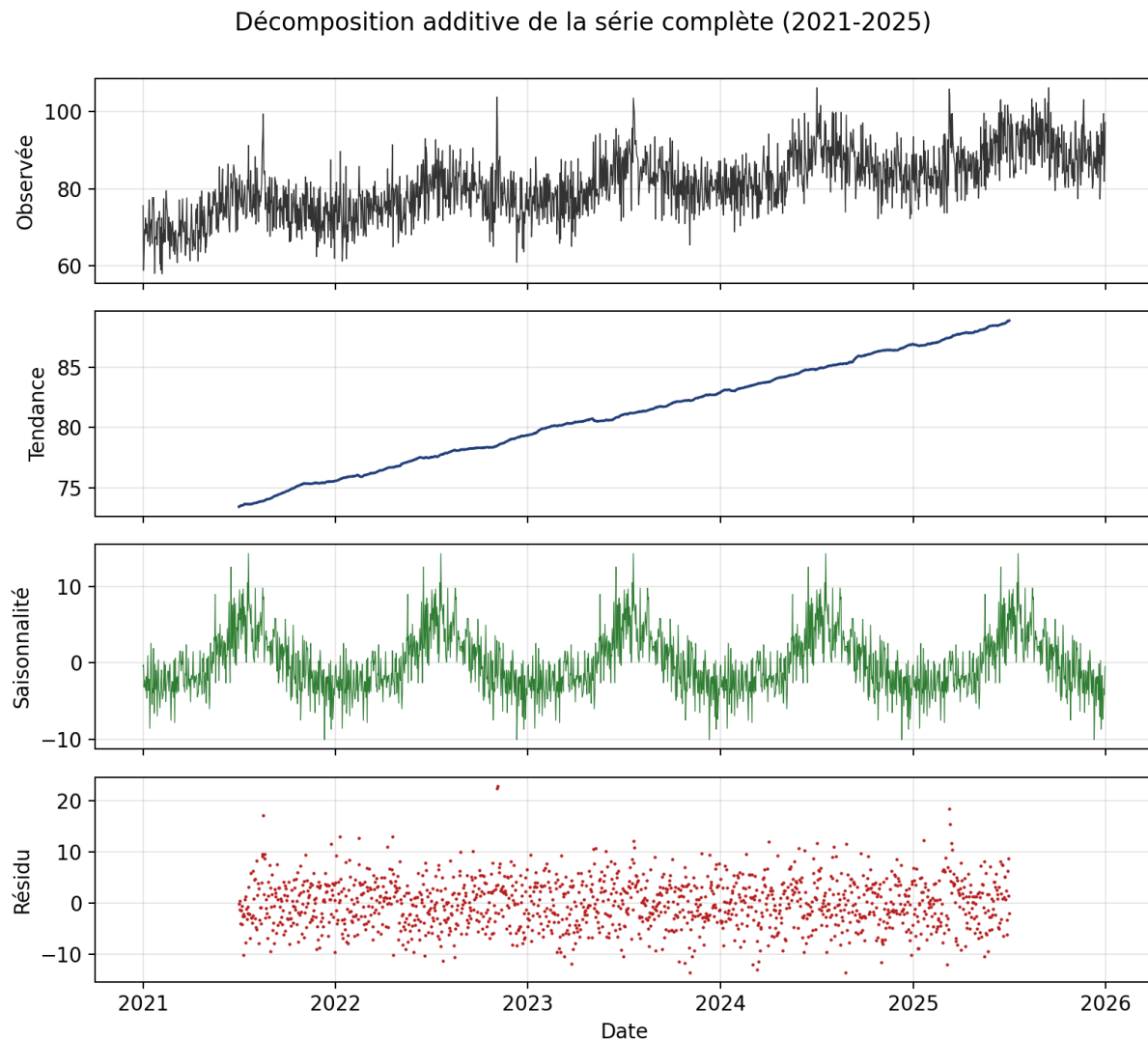


Figure 2: Décomposition additive de la série complète (2021-2025). La tendance capture la croissance lente, la saisonnalité le cycle annuel du paludisme, et le résidu les variations imprévisibles (dont les chocs épidémiques).

```

1 from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
2
3 fig, axes = plt.subplots(1, 2, figsize=(12, 4))
4
5 # lags=40 : on regarde la corrélation jusqu'à 40 jours de décalage
6 plot_acf(df["demande_sang"], lags=40, ax=axes[0])
7 axes[0].set_title("ACF")
8
9 # method="ywm" : méthode de calcul (Yule-Walker modifiée), stable et
10 # recommandée
11 plot_pacf(df["demande_sang"], lags=40, ax=axes[1], method="ywm")
12 axes[1].set_title("PACF")

```

Comment lire ce graphique ? Les barres qui dépassent la zone bleue aux lags 1, 2, 7, 14 confirment ce qu'on attendait : la valeur d'hier (y_{t-1}) et la valeur d'il y a une semaine (y_{t-7}) sont de bons candidats comme **lag features**.

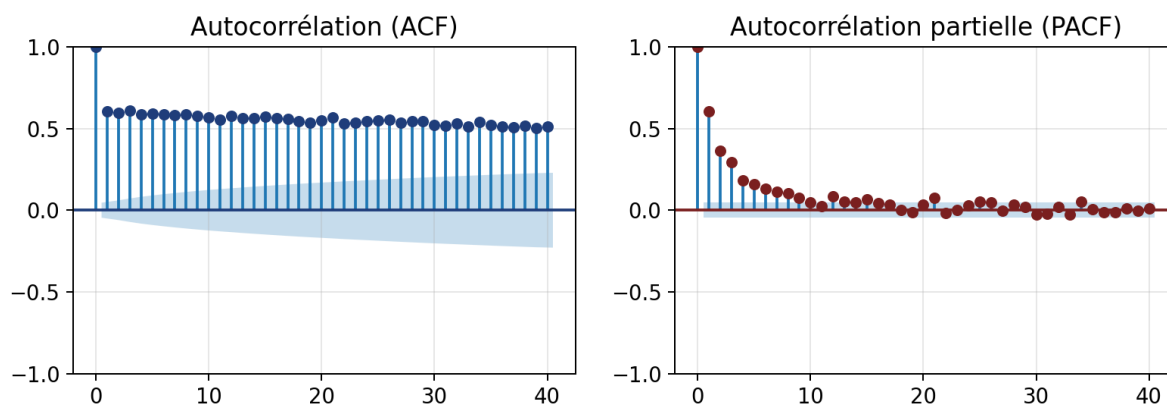


Figure 3: ACF (gauche) et PACF (droite) de la demande de sang. La zone bleue est l'intervalle de confiance à 95% : une barre qui en sort est significative.

7 Feature engineering temporelle

C'est l'étape la plus importante pour préparer tes données à un modèle comme Random Forest ou XGBoost.

Listing 5: Construction des features temporelles

```

1  # --- Features calendaires -----
2  # .dt accede aux composantes d'une colonne datetime
3  df["jour_semaine"] = df["date"].dt.dayofweek    # 0=lundi ... 6=dimanche
4  df["mois"] = df["date"].dt.month                # 1 a 12
5  df["jour_annee"] = df["date"].dt.dayofyear      # 1 a 365/366 (capture la
6  #                                     saisonnalite annuelle)
7  # --- Lag features -----
8  # .shift(k) decale la serie de k lignes vers le bas :
9  # a la ligne t, on retrouve la valeur qui etait a la ligne t-k
10 for lag in [1, 7, 14]:
11     df[f"demande_lag_{lag}"] = df["demande_sang"].shift(lag)
12
13 # --- Statistiques glissantes (rolling) -----
14 # ATTENTION au piege classique : si on fait rolling() directement sur
15 # "demande_sang", la fenetre inclut la valeur du jour courant -> FUITE DE
16 # DONNEES
17 # (le modele "trichereait" en utilisant une information qu'il n'aurait pas
18 # au moment de la prediction reelle). Il faut TOUJOURS decaler d'abord.
19 df["demande_rolling_mean_7"] = df["demande_sang"].shift(1).rolling(window
20     =7).mean()
21 df["demande_rolling_std_7"] = df["demande_sang"].shift(1).rolling(window
22     =7).std()
23
24 # Les premieres lignes contiennent des NaN (pas assez d'historique) : on
25 # les retire
26 df = df.dropna().reset_index(drop=True)

```

Définition — Paramètres importants à retenir

- `shift(k)` : décale la série de k pas ; **toujours** utiliser `shift(1)` avant un `rolling()` pour éviter la fuite de données (*data leakage*).
- `window` (dans `rolling`) : taille de la fenêtre glissante, en nombre d'observations (ici, en jours).
- `min_periods` : nombre minimal d'observations requises pour calculer la statistique (par défaut égal à `window` ; on peut le réduire si on veut des valeurs dès le début de la série).

Exercice 7.1

Ajoute une feature `demande_lag_365` (valeur d'il y a un an, même jour). Est-ce qu'elle est utile ici, sachant que le dataset ne couvre que 5 ans ? Que se passe-t-il pour les 365 premiers jours ?

8 Validation croisée pour les séries temporelles

Erreur à ne jamais commettre

Sur une série temporelle, on **ne fait jamais** un `train_test_split` classique avec mélange aléatoire (`shuffle=True`). Cela reviendrait à entraîner le modèle sur des données du futur pour prédire le passé — un biais optimiste énorme qu'on appelle la **fuite temporelle**.

Listing 6: `TimeSeriesSplit` : validation croisée respectant l'ordre du temps

```
1 from sklearn.model_selection import TimeSeriesSplit
2
3 # n_splits=5 : on cree 5 "plis" successifs. A chaque pli, le jeu
4 # d'entraînement grandit et le jeu de test est TOUJOURS APRES dans le
   temps
5 tscv = TimeSeriesSplit(n_splits=5)
6
7 for fold, (train_idx, test_idx) in enumerate(tscv.split(X)):
8     X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
9     y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]
10    print(f"Pli {fold+1} : entraînement sur {len(train_idx)} jours, "
11          f"test sur {len(test_idx)} jours (toujours dans le futur)")
```

Concrètement, `TimeSeriesSplit` découpe la chronologie ainsi (avec `n_splits=3`) :

	Pli 1	Pli 2	Pli 3
Entraînement	Jours 1–500	Jours 1–1000	Jours 1–1500
Test	Jours 501–1000	Jours 1001–1500	Jours 1501–1826

9 Passer aux modèles d'ensemble

Une fois les features temporelles construites et la validation temporelle en place, tu peux appliquer **exactement** ce que tu vas apprendre sur Random Forest, XGBoost, LightGBM ou CatBoost — sans rien changer côté « série temporelle ».

Listing 7: Exemple complet avec RandomForestRegressor

```

1 from sklearn.ensemble import RandomForestRegressor
2 from sklearn.model_selection import TimeSeriesSplit
3 from sklearn.metrics import mean_absolute_error, mean_squared_error
4
5 features = ["temperature_c", "precipitations_mm", "cas_paludisme",
6            "nb_accidents", "nb_accouchements", "jour_semaine", "mois",
7            "jour_annee", "demande_lag_1", "demande_lag_7", "
8            demande_lag_14",
9            "demande_rolling_mean_7", "demande_rolling_std_7"]
10 X = df[features]
11 y = df["demande_sang"]
12
13 tscv = TimeSeriesSplit(n_splits=5)
14 maes = []
15
16 for train_idx, test_idx in tscv.split(X):
17     X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
18     y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]
19
20     # n_estimators : nombre d'arbres dans la foret (plus = plus stable,
21     # plus lent)
22     # max_depth : profondeur max de chaque arbre (limite le
23     # surapprentissage)
24     # random_state : graine pour la reproductibilite
25     # n_jobs=-1 : utilise tous les coeurs du processeur disponibles
26     model = RandomForestRegressor(
27         n_estimators=300, max_depth=8, random_state=42, n_jobs=-1
28     )
29     model.fit(X_train, y_train)
30     pred = model.predict(X_test)
31
32     mae = mean_absolute_error(y_test, pred) # erreur absolue moyenne (
33     # meme unite que y)
34     rmse = mean_squared_error(y_test, pred) ** 0.5 # penalise davantage
35     # les grosses erreurs
36     maes.append(mae)
37     print(f"MAE={mae:.2f} RMSE={rmse:.2f}")
38
39 print("MAE moyenne sur les 5 plis :", sum(maes) / len(maes))

```

Sur ce dataset, ce code donne une **MAE moyenne d'environ 4.7**, ce qui signifie que le modèle se trompe en moyenne de moins de 5 unités sur un indice qui varie entre 58 et 106 — un très bon point de départ.

Définition — MAE vs RMSE

- **MAE** (*Mean Absolute Error*) : moyenne des erreurs absolues. Facile à interpréter, dans la même unité que la cible.
- **RMSE** (*Root Mean Squared Error*) : racine carrée de la moyenne des erreurs au carré. Pénalise davantage les grosses erreurs (utile si les pics de pénurie sont particulièrement critiques à bien prédire).

Exercice 9.1

Remplace `RandomForestRegressor` par `XGBRegressor` (à installer avec `pip install xgboost`). Compare la MAE obtenue.

Exercice 9.2

Ajoute une feature `alerte_risque` qui vaut 1 si `demande_sang` dépasse le 90^e percentile de la fenêtre glissante des 30 derniers jours, 0 sinon. Transforme ainsi le problème en classification (« y aura-t-il un risque de pénurie dans 7 jours ? ») plutôt qu'en régression.

10 Exercices de synthèse

Exercice 10.1 — Décalage de la cible pour prédire à J+7

Ton projet vise à alerter **7 jours avant** qu'un stock devienne critique. Cela veut dire que la cible du modèle ne doit pas être `demande_sang` du jour même, mais sa valeur **dans 7 jours** :

```
1 df["cible_j7"] = df["demande_sang"].shift(-7)
```

Attention au sens du décalage ! Ici on utilise un `shift` négatif car on veut une valeur **future**, pas passée. Adapte le pipeline du chapitre 9 pour prédire `cible_j7` à partir des features disponibles **aujourd'hui**.

Exercice 10.2 — Comparaison des quatre modèles d'ensemble

Une fois `Random Forest`, `XGBoost`, `LightGBM` et `CatBoost` maîtrisés individuellement, compare-les sur ce même dataset avec le même `TimeSeriesSplit`, et construis un tableau MAE/RMSE pour les quatre.

Exercice 10.3 — Vers les vraies données

Remplace ce dataset synthétique par tes vraies données (météo Open-Meteo, statistiques d'accidents/accouchements/maladies par région togolaise). Est-ce que la saisonnalité et les lags optimaux restent les mêmes ?

11 Pour aller plus loin

- **Documentation statsmodels** : <https://www.statsmodels.org/stable/tsa.html>
- **Documentation scikit-learn (TimeSeriesSplit)** : https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.TimeSeriesSplit.html
- Une fois ce chapitre maîtrisé, tu es prêt à attaquer `Random Forest`, `XGBoost`, `LightGBM` et `CatBoost` en profondeur — tu réutiliseras directement les features construites ici.

Bon courage pour la suite, pote — LAFIè IA va bénéficier directement de tout ce que tu viens d'apprendre.